

IC 4301 Bases de Datos I

Tercera Tarea Programada

Control de Asistencia y Planilla Obrera

Instituto Tecnológico de Costa Rica
Escuela de Computación

Integrantes:

Fernando Abarca Aguilar
Isaac Van der Bom López

Profesor: Franco Quirós Ramírez
Fecha: 18 Junio 2026

Capítulo 1. Introducción	3
1.1 Objetivo general	3
1.2 Objetivos específicos	3
Capítulo 2. Ambiente y arquitectura de aplicación	4
2.1 Ambiente de desarrollo	4
Figura 1. Diagrama del ambiente de desarrollo y ejecución.	4
2.2 Arquitectura de aplicación	5
Figura 2. Arquitectura de aplicación.	5
2.3 Modelo general de base de datos	6
Figura 3. Organización lógica del modelo de base de datos.	6
Capítulo 3. Resultados del proyecto	7
Tabla 1. Métricas del proyecto	7
Capítulo 4. Métricas del proyecto	12
Tabla 2. Métricas generales del proyecto.	12
Figura 4. Distribución de marcas de asistencia por empleado.	15
Tabla 3. Resultados de auditoría final de base de datos.	16
Capítulo 5. Evidencias y decisiones relevantes	17
Capítulo 6. Conclusiones	18
Capítulo 7. Referencias Bibliográficas	18

Capítulo 1. Introducción

El presente documento constituye el análisis de resultados de la Tercera Tarea Programada del curso IC 4301 Bases de Datos I. El proyecto consiste en el diseño e implementación de un sistema de Control de Asistencia y Planilla Obrera sobre Microsoft SQL Server, con una aplicación web complementaria desarrollada en Flask. El sistema permite administrar empleados, registrar asistencia, procesar horas ordinarias y extraordinarias, calcular deducciones, generar planillas semanales y mensuales, y mantener trazabilidad mediante bitácora y manejo centralizado de errores.

El objetivo del proyecto fue construir una solución funcional basada en procedimientos almacenados, archivos XML de entrada, reglas de negocio en base de datos y una interfaz web para dos tipos de usuario: administrador y empleado. Entre los requerimientos principales se incluyen la creación del modelo relacional, la carga de catálogos desde XML, la simulación completa de operaciones, el cálculo de planilla, el registro de eventos en bitácora, el manejo de errores en DBErrors y las consultas web para visualizar empleados, planillas y detalles de deducciones u horas.

El documento se organiza en seis capítulos. El Capítulo 1 introduce el proyecto, sus objetivos y el alcance general. El Capítulo 2 describe el ambiente tecnológico, la arquitectura de aplicación y la estructura general de la base de datos. El Capítulo 3 presenta los resultados obtenidos frente a los elementos calificables de la tarea. El Capítulo 4 resume métricas cuantitativas del desarrollo y de la ejecución final. El Capítulo 5 documenta evidencias y decisiones técnicas relevantes. Finalmente, el Capítulo 6 presenta las conclusiones del trabajo realizado.

1.1 Objetivo general

Desarrollar un sistema de Control de Asistencia y Planilla Obrera que permita procesar, mediante SQL Server y una aplicación web, la información de empleados, jornadas, asistencias, deducciones y planillas semanales y mensuales, cumpliendo con los requerimientos funcionales, de trazabilidad y de estándar de codificación definidos para la tercera tarea programada.

1.2 Objetivos específicos

1. Diseñar una base de datos relacional normalizada para empleados, usuarios, catálogos, asistencia, deducciones, movimientos y planillas.
2. Implementar scripts SQL reproducibles para crear la base de datos, cargar catálogos desde XML y ejecutar la simulación completa desde el XML de operaciones.
3. Centralizar la lógica de negocio en procedimientos almacenados, evitando SQL incrustado en la aplicación web.
4. Calcular correctamente horas ordinarias, horas extra normales, horas extra dobles, deducciones obligatorias, deducciones opcionales, salario bruto y salario neto.
5. Registrar eventos relevantes en bitácora y errores en la tabla DBErrors.
6. Proveer una interfaz web funcional para administrador y empleado, con login, listado y filtro de empleados, impersonación, consulta de planilla semanal, consulta de planilla mensual y detalles asociados.
7. Verificar la consistencia final de los datos cargados desde XML mediante pruebas de integridad, comparaciones contra la base de datos y revisión de acumulados.

Capítulo 2. Ambiente y arquitectura de aplicación

Este capítulo describe el ambiente tecnológico utilizado para el desarrollo y ejecución del sistema. Además, se explica la arquitectura de aplicación, la forma en que la Web se conecta a SQL Server y la organización general del modelo de datos. Los diagramas incluidos permiten visualizar la separación de responsabilidades entre navegador, aplicación Flask, procedimientos almacenados y base de datos.

2.1 Ambiente de desarrollo

En este punto mencionaremos como es el ambiente de desarrollo utilizado en esta tarea programada, adjuntando una imagen. El ecosistema tecnológico seleccionado para la implementación y validación de esta tarea programada combina herramientas locales de edición, control de versiones y gestión de datos. Esta infraestructura asegura un entorno reproducible y estandarizado para la ejecución de scripts y el desarrollo de la lógica de negocio.

El desarrollo se realizó en estaciones de trabajo locales utilizando Visual Studio Code como editor principal, Git como sistema de control de versiones y SQL Server como motor de base de datos. La conexión a la base de datos se realiza por medio de ODBC Driver 18 for SQL Server y la librería pyodbc desde Python. Para la ejecución de scripts y pruebas se utilizó sqlcmd contra la instancia local de SQL Server.

En la Figura 1 se representa el ambiente de desarrollo y ejecución. El navegador accede a la aplicación Flask, Flask se conecta a SQL Server mediante pyodbc/ODBC, y SQL Server ejecuta exclusivamente procedimientos almacenados, funciones, trigger y restricciones definidas en la base de datos PlanillaObrera.

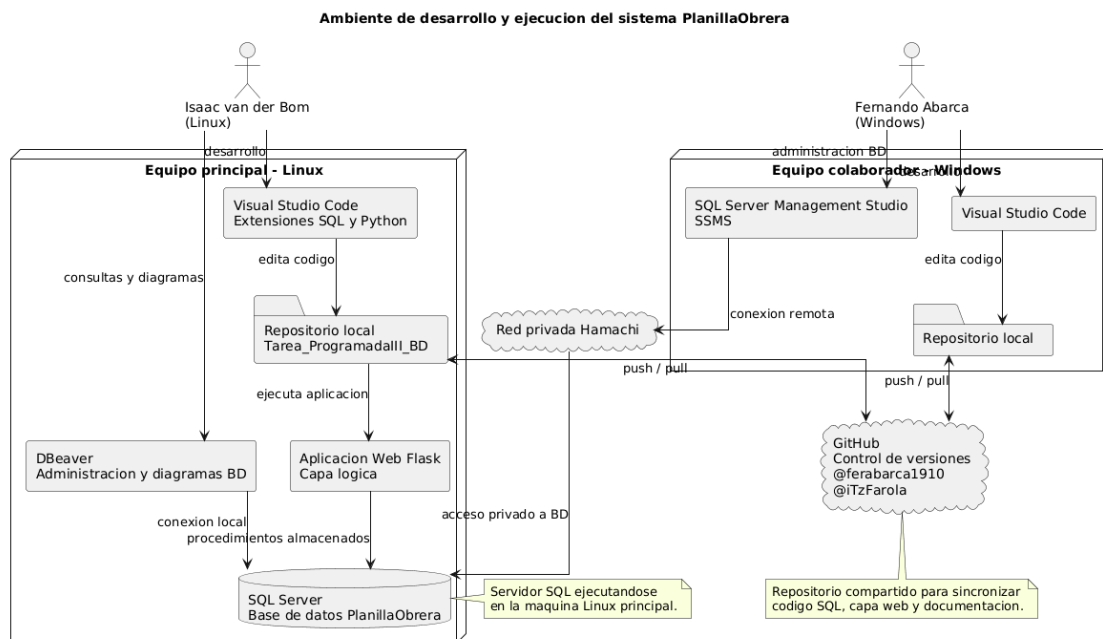


Figura 1. Diagrama del ambiente de desarrollo y ejecución.

2.2 Arquitectura de aplicación

En el presente punto se hablará de la arquitectura con la que cuenta la aplicación además de una imagen representativa de la arquitectura. La solución de software se diseñó bajo un enfoque desacoplado que separa de forma estricta la interfaz de usuario, la gestión de rutas y la persistencia de datos. Esta organización estructural optimiza el flujo de información y garantiza que la lógica operativa quede centralizada y protegida dentro del entorno del motor de base de datos.

La solución utiliza una arquitectura por capas con una organización cercana al patrón MVC. Las plantillas HTML con Jinja2 funcionan como la vista, los módulos de rutas de Flask funcionan como controladores y los procedimientos almacenados de SQL Server concentran el modelo de datos y la lógica de negocio. Esta decisión evita que la aplicación web contenga SQL directo y facilita que las reglas de planilla se mantengan centralizadas en la base de datos.

La Figura 2 muestra la arquitectura de aplicación. El usuario interactúa con formularios y pantallas web; Flask valida sesión, decide la ruta correspondiente y llama procedimientos almacenados mediante wrappers como `sp_WebLogin`, `sp_WebListarEmpleadosConFiltro` y `sp_WebConsultarPlanillaSemanal`. Los SP internos ejecutan la lógica de administración, simulación, consultas, bitácora y manejo de errores.

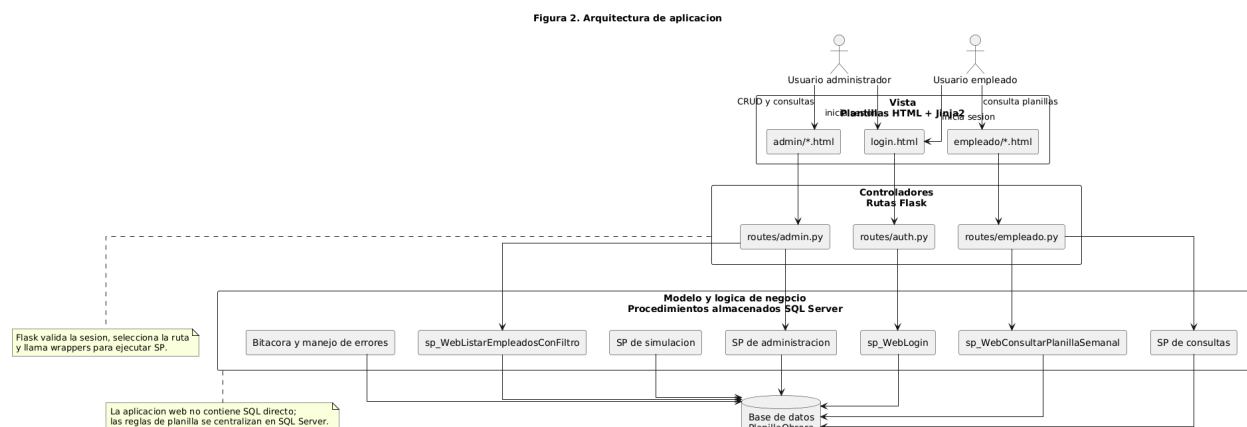


Figura 2. Arquitectura de aplicación.

2.3 Modelo general de base de datos

En este punto se hará énfasis en el modelo general de la bases de datos , además de que se hará una explicación se adjuntará una imagen para una mejor ayuda visual. El diseño relacional del sistema PlanillaObrera está estructurado de manera modular para garantizar la integridad y consistencia de la información laboral. El modelo se organiza en componentes específicos que abarcan desde catálogos maestros y operaciones periódicas hasta submódulos especializados en la trazabilidad de eventos y auditoría de fallos.

La base de datos PlanillaObrera está compuesta por 20 tablas. Estas se agrupan en catálogos, tablas operativas, tablas de planilla y tablas de control. Los catálogos se cargan desde catalogos.xml y contienen jornadas, puestos, feriados, tipos de movimiento, tipos de deducción, tipos de evento y códigos de error. Las tablas operativas almacenan usuarios, empleados, deducciones, jornadas por semana y marcas de asistencia. Las tablas de planilla almacenan periodos semanales, periodos mensuales, movimientos y acumulados por empleado. Finalmente, BitacoraEvento y DBErrors registran trazabilidad y errores técnicos.

La Figura 3 resume la organización lógica del modelo. Es importante indicar que, por aclaración del profesor, las tablas TipoDocumentoidentidad y Departamento no fueron incluidas. Por esta razón, Empleado utiliza ValorDocumentoidentidad como un VARCHAR simple y no posee llaves foráneas hacia esos catálogos.

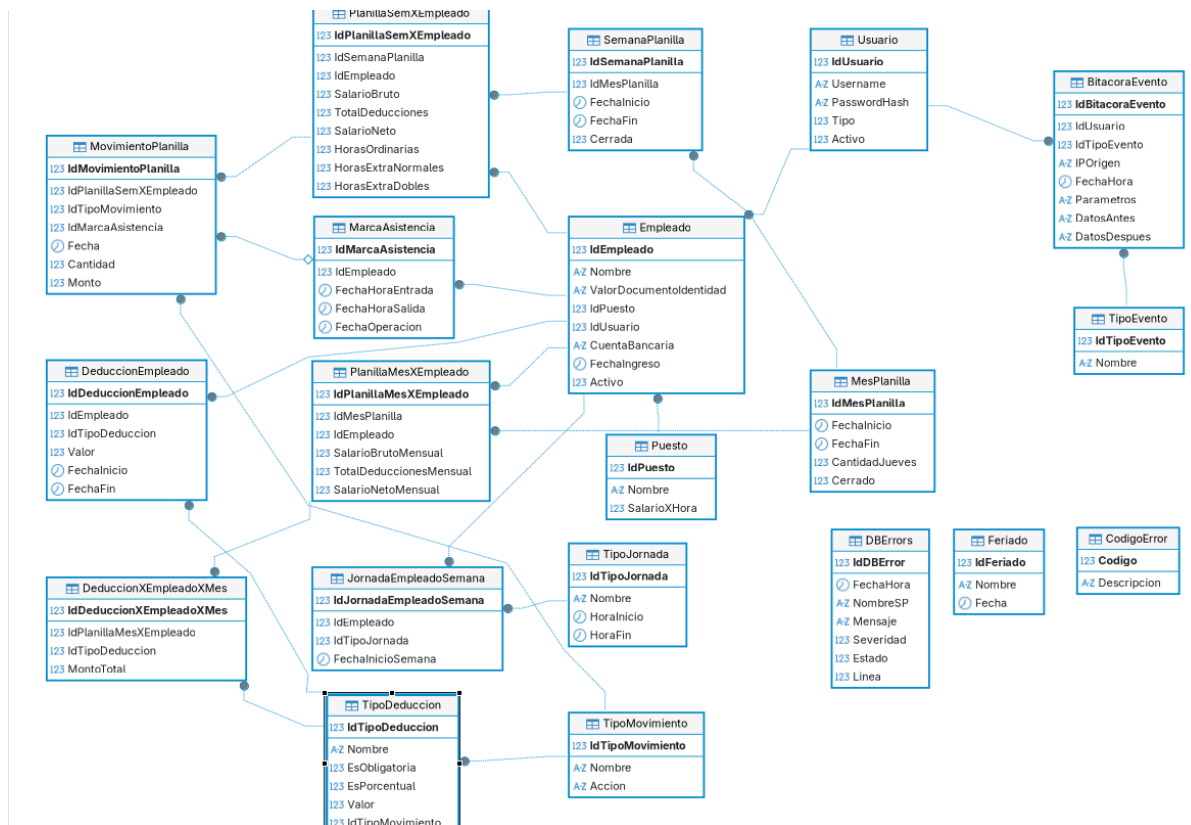


Figura 3. Organización lógica del modelo de base de datos.

Capítulo 3. Resultados del proyecto

Este capítulo presenta el estado final de los elementos evaluables de la tarea. La Tabla 1 se construyó a partir de los requerimientos funcionales y técnicos del proyecto, considerando el resultado final después de ejecutar los scripts desde cero con catalogos.xml y operacion.xml. Cada resultado incluye el porcentaje de cumplimiento y un comentario breve del estado alcanzado.

Tabla 1. Métricas del proyecto

Ítem	Resultado
Documento de análisis de resultados	100%. Se incluye portada, índice de contenido, índice de figuras, introducción, objetivos, ambiente, arquitectura, resultados, métricas, gráficos, evidencias y conclusiones.
Creación de base de datos	100%. El script dll.sql crea la base PlanillaObrera con 20 tablas, restricciones PK, FK, UNIQUE y CHECK.
Modelo de datos ajustado a la aclaración del profesor	100%. No se incluyen TipoDocumentoidentidad ni Departamento. Empleado mantiene ValorDocumentoidentidad como VARCHAR sin FK a esos catálogos.
Carga de catálogos desde XML	100%. catalogos.xml se procesa mediante sp_catalogos.sql y ejecutar_catalogos.sql con ResultCode 0.
Trigger de deducciones obligatorias	100%. trg_AsociarDeducciones asocia automáticamente las deducciones obligatorias al insertar empleados.

Ítem	Resultado
SPs administrativos	100%. sp_InsertarEmpleado, sp_EliminarEmpleado, sp_AsociarDeduccion y sp_DesasociarDeduccion funcionan con parámetros @in/@out, TRY/CATCH, transacciones y bitácora.
Inserción de empleados desde XML	100%. El XML contiene 13 inserciones y la base contiene 13 empleados.
Eliminación lógica de empleados	100%. El XML contiene 4 eliminaciones y la base queda con 9 empleados activos y 4 inactivos.
Asociación de deducciones opcionales	100%. El XML contiene 20 asociaciones; la tabla DeduccionEmpleado conserva historial y fechas.
Desasociación de deducciones	100%. El XML contiene 6 desasociaciones; se registra FechaFin en lugar de borrar físicamente.
Procesamiento de marcas de asistencia	100%. El XML contiene 560 marcas y la base contiene 560 marcas insertadas, sin faltantes ni sobrantes.
Procesamiento de jornadas	100%. El XML contiene 106 asignaciones y la base contiene 106 registros en JornadaEmpleadoSemana.

Ítem	Resultado
Cálculo de horas ordinarias y extra	100%. sp_ProcesarAsistencia calcula horas ordinarias, extra normales y extra dobles considerando jornadas, domingos y feriados.
Cierre y apertura semanal	100%. Se generan 13 semanas de planilla y 103 planillas semanales por empleado.
Cierre y apertura mensual	100%. Se generan 4 meses de planilla y 37 planillas mensuales por empleado.
Movimientos de planilla	100%. Se generan 974 movimientos de planilla; los acumulados semanales coinciden con los movimientos.
Deducciones mensuales	100%. Se generan acumulados por tipo de deducción y los montos mensuales coinciden con la suma semanal.
Manejo de errores	100%. DBErrors queda en 0 después de la ejecución final. Los CATCH registran errores técnicos cuando ocurren.
Bitácora de eventos	100%. Se registran eventos de carga, simulación, login, consultas, administración e impersonación.
Login Web	100%. sp_WebLogin valida usuarios administrador y empleado. admin/admin123

Ítem	Resultado
	y Mencar/Gojira devuelven ResultCode 0.
Pantalla administrador	100%. Permite listar empleados activos, filtrar por nombre/documento, editar, crear, eliminar e impersonar empleados.
Pantalla empleado semanal	100%. Muestra últimas planillas semanales, salario bruto, deducciones, salario neto, horas ordinarias y horas extra. Bruto y deducciones tienen detalle.
Pantalla empleado mensual	100%. Muestra planillas mensuales, bruto, deducciones, neto y detalle mensual de deducciones.
Separación de responsabilidades SQL	100%. sp_simulacion.sql contiene simulación, sp_consultas.sql contiene consultas, sp_admin.sql administración y sp_catalogos.sql carga de catálogos.
Ausencia de SQL directo en Web	100%. La Web llama procedimientos almacenados mediante pyodbc y wrappers sp_Web*.
Estándar SQL	100%. Se utiliza dbo, parámetros @in/@out, SET NOCOUNT ON, TRY/CATCH, DBErrors, aliases, transacciones donde aplica y no se usan cursores.

Ítem	Resultado
Prueba final desde cero	100%. La ejecución completa desde scripts oficiales finaliza con ResultCodeInicializar 0, ResultCodeSimulacion 0 y DBErrors 0.

Capítulo 4. Métricas del proyecto

Este capítulo presenta métricas cuantitativas del proyecto. La Tabla 2 resume datos de tamaño, esfuerzo, volumen de código, cantidad de objetos de base de datos, datos procesados y resultados de prueba. Estas métricas permiten dimensionar la complejidad real de la implementación y respaldan el análisis de cumplimiento presentado en el capítulo anterior.

Tabla 2. Métricas generales del proyecto.

Con el propósito de cuantificar el esfuerzo técnico y registrar la evolución del proyecto, se recopilieron los indicadores clave del repositorio de control de versiones y de la planificación temporal. La Tabla 2 detalla el cronograma efectivo, el volumen de contribuciones distribuidas por el equipo y la inversión estimada de horas de ingeniería.

Nombre de la métrica	Valor	Comentarios
Fecha de inicio registrada en Git	13 de mayo de 2026	Primer commit identificado en el historial local.
Fecha de cierre de pruebas	18 de junio de 2026	Fecha de la auditoría final con los XML oficiales.
Commits en Git	98	Historial distribuido entre mayo y junio de 2026.(49 Fernando-47 Isaac)
Total de horas estimadas	95 horas	Incluye diseño, SQL, Web, pruebas, depuración y documentación.
Tablas creadas	20	Incluye catálogos, tablas operativas, planilla, bitácora y errores.

Nombre de la métrica	Valor	Comentarios
Procedimientos almacenados	43	Distribuidos en sp_catalogos, sp_admin, sp_simulacion y sp_consultas.
Funciones SQL	3	Funciones auxiliares para feriados/domingos, último jueves y conteo de jueves.
Trigger	1	trg_AsociarDeducciones.
Líneas SQL principales	4141	Conteo de bd/*.sql y bd/sp/*.sql.
Líneas Web principales	1524	Conteo de Python, templates HTML, CSS y JS.
Líneas totales contadas	5669	Incluye SQL, Web, README y requirements.
Operaciones FechaOperacion en XML	85	Cantidad de nodos FechaOperacion en operacion.xml.
Empleados insertados por XML	13	Coincide con la cantidad de empleados en BD.

Nombre de la métrica	Valor	Comentarios
Empleados activos finales	9	Cuatro empleados quedan inactivos por eliminación lógica.
Marcas de asistencia procesadas	560	Coincide XML vs BD.
Asignaciones de jornada procesadas	106	Coincide XML vs BD.
Semanas de planilla generadas	13	De marzo a inicio de junio 2026.
Planillas semanales por empleado	103	Registros en PlanillaSemXEmpleado.
Meses de planilla generados	4	Marzo, abril, mayo y apertura de junio 2026.
Planillas mensuales por empleado	37	Registros en PlanillaMesXEmpleado.
Movimientos de planilla	974	Créditos y débitos generados por asistencia y deducciones.
Errores finales en DBErrors	0	La ejecución oficial queda

Nombre de la métrica	Valor	Comentarios
		sin errores.

La Figura 4 muestra la distribución de marcas de asistencia por empleado al finalizar la simulación. Evidencia que los empleados con permanencia completa en el periodo tienen más marcas, mientras que los empleados insertados tarde o eliminados durante la simulación poseen menos registros, lo cual concuerda con el comportamiento esperado del XML.

	Empleado	CantidadMarcas
1	Nicolas Vargas	72
2	Laura Castro	72
3	Pedro Arias	72
4	Carlos Mendoza	72
5	Sofia Mora	66
6	Gabriela Leon	54
7	Valeria Nunez	30
8	Roberto Fallas	30
9	Diego Solano	29
10	Andres Vega	23
11	Jimena Salazar	18
12	Ana Rodriguez	17
13	Mario Quesada	5

Figura 4. Distribución de marcas de asistencia por empleado.

Tabla 3. Resultados de auditoría final de base de datos.

Para validar la correcta migración de la información y asegurar la ausencia de inconsistencias lógicas, se ejecutó un protocolo riguroso de conciliación de datos. Los resultados del proceso de auditoría y los comandos de verificación de restricciones sobre la base de datos se consolidan en la Tabla 3.

Prueba	Resultado
Empleados XML vs BD	0 diferencias.
Marcas XML vs BD	0 diferencias.
Jornadas XML vs BD	0 diferencias.
DBErrors	0 errores finales.
Planillas sin empleado	0 casos.
Acumulados semanales vs movimientos	0 diferencias.
Acumulados mensuales vs semanas	0 diferencias.
DBCC CHECKCONSTRAINTS	Sin errores reportados.

Capítulo 5. Evidencias y decisiones relevantes

Este capítulo resume decisiones de diseño y evidencias técnicas que respaldan el resultado final. Estas decisiones fueron tomadas para mejorar la mantenibilidad, cumplir el estándar de codificación y facilitar la defensa del proyecto durante la revisión.

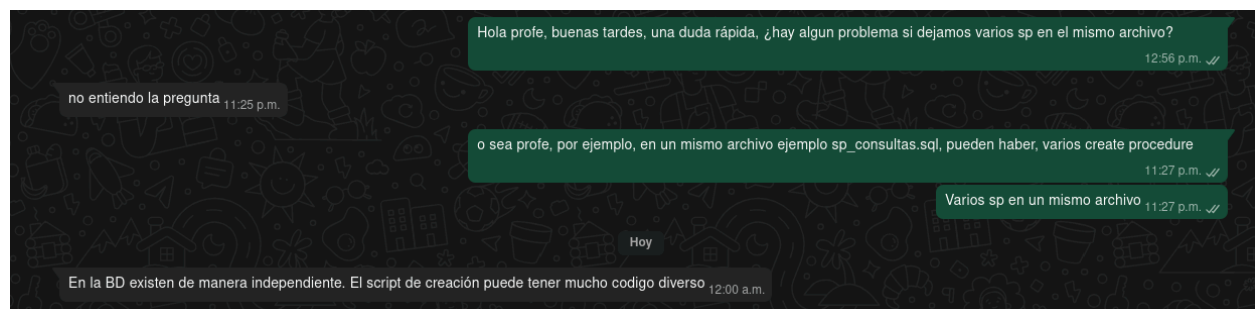
La primera decisión relevante fue separar los scripts SQL por responsabilidad. El archivo dll.sql contiene únicamente el modelo de base de datos; sp_catalogos.sql contiene la carga de catálogos; sp_admin.sql contiene operaciones administrativas; sp_simulacion.sql contiene la simulación; y sp_consultas.sql contiene las consultas requeridas por la Web. Esta separación reduce el acoplamiento y facilita ejecutar pruebas parciales.

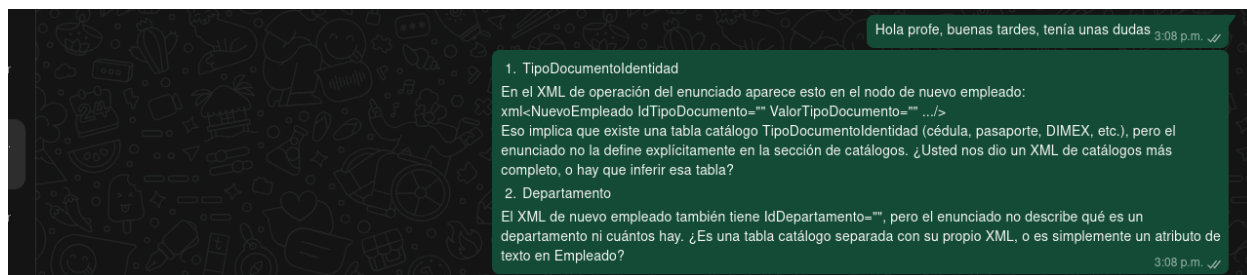
La segunda decisión fue mover toda la lógica de negocio a SQL Server. La Web no contiene SQL directo y todas sus operaciones se canalizan por procedimientos almacenados. Esto permite mantener control sobre transacciones, validaciones, bitácora y errores desde la base de datos.

La tercera decisión fue conservar historial en lugar de borrar datos operativos. Las eliminaciones de empleados son lógicas mediante el campo Activo, y las desasociaciones de deducciones se registran con FechaFin. Esta decisión permite que las planillas históricas sigan siendo auditable aun cuando un empleado ya no esté activo.

La cuarta decisión fue utilizar XML como fuente oficial de datos de prueba. catalogos.xml define los datos base y operacion.xml define el flujo completo de simulación. La auditoría final demostró que los registros principales del XML coinciden contra la base de datos sin diferencias.

Como evidencia de ejecución, se verificó que los scripts oficiales pueden correrse desde cero en el orden esperado. Además, se probaron los SPs de login, consultas de administrador, consultas de empleado, impersonación y detalles de planilla. También se verificó que no se utilizan cursores ni SELECT * en los archivos SQL y Python del proyecto.





Enlace de la Bitácora: [ProyectoIII BD](#)

Capítulo 6. Conclusiones

El proyecto cumple con los requerimientos funcionales principales de la Tercera Tarea Programada. La base de datos se crea correctamente, los catálogos se cargan desde XML, la simulación procesa empleados, jornadas, marcas de asistencia, deducciones y planillas, y la aplicación web permite consultar la información desde los roles de administrador y empleado. La auditoría final evidencia que los datos cargados desde el XML oficial concuerdan con la base de datos. No se detectaron diferencias entre empleados, marcas o jornadas; tampoco se detectaron errores finales en DBErrors, duplicados críticos, registros huérfanos ni diferencias entre acumulados semanales, mensuales y movimientos de planilla.

Desde el punto de vista de arquitectura, la solución mantiene una separación clara entre presentación, controladores web, procedimientos almacenados y datos. El uso de procedimientos almacenados como frontera entre Web y base de datos fortalece la integridad del sistema y facilita la validación del estándar solicitado. En conjunto, el proyecto queda en condiciones adecuadas para su ejecución, demostración y revisión.

Capítulo 7. Referencias Bibliográficas

Ben-Gan, I. (2016). T-SQL Fundamentals (3.^a ed.). Microsoft Press.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7.^a ed.). McGraw-Hill Education.

Microsoft Learn. (2023). Create a Stored Procedure. <https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/create-a-stored-procedure>

Microsoft Learn. (2022). Examples: Bulk Import and Export of XML Documents. <https://learn.microsoft.com/en-us/sql/relational-databases/import-export/examples-bulk-import-and-export-of-xml-documents-sql-server>

W3Schools. (s.f.). SQL Stored Procedures for SQL Server. https://www.w3schools.com/sql/sql_stored_procedures.asp